



Desenvolvimento Front-End

Prof. Elias A. da Silva
elias.asilva@sp.senac.br

HTML – Áudio

<audio controls>

<source src="audio.mp3" type="audio/mpeg">

<source src="audio.ogg" type="audio/ogg">

Seu navegador não suporta áudio HTML5.

</audio>

Mesmos atributos do video: controls, autoplay, muted, loop, preload.

Sem width e height, sem poster.

Muito usado em podcasts e players de música.

HTML – Áudio

Áudio segue a mesma lógica do vídeo. Multiple source é fallback de formato. O texto dentro da tag aparece só em navegadores muito antigos que não suportam a tag, hoje isso é raridade. Se quiserem testar, coloquem qualquer MP3 na pasta e apontem o src.



HTML – Áudio

Digitem abaixo do código de vídeo:

```
<h3>Áudio</h3>
```

```
<figure>
```

```
  <audio controls preload="metadata">
```

```
    <source src="audios/demo.mp3" type="audio/mpeg">
```

```
    Seu navegador não suporta reprodução de áudio.
```

```
  </audio>
```

```
<figcaption>Faixa de demonstração.</figcaption>
```

```
</figure>
```



HTML – IFrame

```
<iframe src="URL" width="600" height="400" title="Descrição  
do conteúdo" loading="lazy" allowfullscreen></iframe>
```

Usos comuns: Vídeo do YouTube, Mapa do Google Maps, Documento PDF, Google Forms.

title é obrigatório → acessibilidade

loading="lazy" → carrega só quando visível na tela



HTML – IFrame

Iframe é o jeito de embutir qualquer conteúdo externo. O YouTube, o Google Maps e a maioria dos serviços geram o código **iframe** automaticamente, você só copia e cola. O atributo **title** é obrigatório para acessibilidade, sem ele o leitor de tela não consegue descrever o conteúdo para o usuário. **Loading lazy** adia o carregamento até o **iframe** ficar visível, o que melhora a performance da página.

HTML – IFrame

Abram o YouTube, escolham qualquer vídeo. Cliquem em Compartilhar, depois Incorporar. Copiem o iframe gerado e colem no código de vocês, vocês vão criar essa área logo abaixo de áudio.



HTML – IFrame

```
<h3>Vídeo do YouTube</h3>
<figure>
  <iframe
    width="560" height="315"
    src="https://www.youtube.com/embed/ID_DO_VIDEO"
    title="Título do vídeo escolhido pela turma"
    loading="lazy"
    allowfullscreen>
  </iframe>
  <figcaption>Vídeo escolhido pela turma.</figcaption>
</figure>

<h3>Mapa</h3>
<figure>
  <iframe
    src="https://www.google.com/maps/embed?pb=CODIGO_DO_MAPS"
    width="560" height="300"
    title="Localização do SENAC Rio Claro"
    loading="lazy"
    allowfullscreen>
  </iframe>
  <figcaption>SENAC Rio Claro, SP.</figcaption>
</figure>
```

Senac

HTML – IFrame

Para o Maps: abram o Google Maps, pesquisem SENAC Rio Claro, cliquem em Compartilhar, depois Incorporar um mapa, copiem o iframe.



HTML – Picture

```
<picture>  
  <source media="(min-width: 800px)" srcset="imagem-  
grande.jpg">  
  <source media="(min-width: 400px)" srcset="imagem-  
media.jpg">  
    
</picture>
```

HTML – Picture

Por que usar?

- Mobile carrega versão menor, economiza dados;
- Desktop carrega versão de alta resolução;
- O navegador escolhe o arquivo correto;
- img é o fallback obrigatório, o alt fica nele;



HTML – Picture

`img` com **max-width** 100% faz a imagem redimensionar visualmente, mas o arquivo baixado continua sendo o mesmo pesado. **Picture** muda o arquivo que é baixado dependendo da tela. Um usuário de celular com 4G não precisa baixar uma imagem de 2000px. **Picture** é performance e responsividade real. O **img** dentro do **picture** é obrigatório, é o **fallback** e é onde o **alt** fica.

HTML – Picture

Digitem o seguinte código, abaixo do IFrame:

```
<h3>Imagem Responsiva</h3>
<figure>
  <picture>
    <source media="(min-width: 800px)" srcset="imagens/banner-desktop.jpg">
    <source media="(min-width: 400px)" srcset="imagens/banner-tablet.jpg">
    
  </picture>
  <figcaption>Redimensione a janela e recarregue, o arquivo carregado muda.</figcaption>
</figure>
```

Os arquivos podem não existir ainda, tudo bem. O objetivo é ver a estrutura. Na prática vocês teriam as três versões preparadas com tamanhos diferentes.

HTML – Inventário

Estrutura: DOCTYPE, html, head, body, meta, title, link

Semântica: header, nav, main, section, article, aside, footer, figure, figcaption, time, address, details, summary

Conteúdo: h1–h6, p, a, img, strong, em, br, hr, blockquote, cite, code, pre, mark, del, ins, small

Listas: ul, ol, li, dl, dt, dd

Formulário: form, label, input (todos os tipos), textarea, select, optgroup, option, datalist, fieldset, legend, button

Tabela: table, caption, thead, tbody, tfoot, tr, th, td

Mídia: video, audio, source, iframe, Picture

Entidades: © — < > &

CSS

Antes do CSS, estilo ficava dentro do HTML:

```
<font color="red" size="5">texto</font>
```

```
<center>conteúdo</center>
```

```
<table bgcolor="#000000">...
```

Problema: estrutura e estilo misturados.

Manutenção impossível em sites grandes.

CSS separou os dois:

HTML → o que é o conteúdo;

CSS → como o conteúdo aparece;

Senac

Anatomia do CSS

```
seletor {  
  propriedade: valor;  
  propriedade: valor;  
}
```

```
p {  
  color: #333333;  
  font-size: 16px;  
  line-height: 1.6;  
}
```

Seletor → para quem a regra se aplica;

Propriedade → o que muda

Valor → como fica;

Ponto e vírgula → obrigatório ao final de cada declaração;

Chaves → agrupam as declarações do seletor;

Vínculos de CSS

1. **EXTERNO** — arquivo separado (sempre use)
No **head**: `<link rel="stylesheet" href="style.css">`
2. **INTERNO** — dentro do **head** (casos específicos)
`<style> p { color: red; } </style>`
3. **INLINE** — direto na tag (evite)
`<p style="color: red;">texto</p>`

Regra profissional: sempre externo. Um arquivo CSS serve múltiplas páginas. Mudar uma cor muda o site inteiro de uma vez.

Vínculos de CSS

No VS Code, na pasta do portfólio(Aula 04), criem style.css. No head do index.html:

```
<link rel="stylesheet" href="style.css">
```

No style.css:

```
p { color: red;}
```

Salvem os dois arquivos. Vejam no Live Server. Todos os parágrafos ficaram vermelhos. Troquem para blue, salvem. Vejam a mudança em tempo real. Deletem esse teste, foi só para confirmar que o link funciona.

CSS – Cascata e herança

CASCADE: regras posteriores sobrescrevem anteriores.

```
p { color: red; }
```

```
p { color: blue; }
```

→ resultado: azul

CSS – Cascata e herança

HERANÇA: propriedades passam do pai para os filhos.

```
body { font-family: Arial; }
```

→ todos os filhos herdam a fonte

O que herda: color, font-family, font-size, line-height, text-align

O que **NÃO** herda: margin, padding, border, background, width

CSS – Seletores

ELEMENTO	<code>p { }</code>	toda tag p da página
CLASSE	<code>.destaque { }</code>	elementos com <code>class="destaque"</code>
ID	<code>#header { }</code>	elemento com <code>id="header"</code>
UNIVERSAL	<code>* { }</code>	todos os elementos

Classe vs ID:

ID → deve ser único por página, use para seções únicas e âncoras

Classe → reutilizável, use para padrões que se repetem

Na prática: classe é usada em 95% dos casos de estilo.

CSS – Seletores – Prática

Dentro de style.css, apaguem o que tiver lá, e digitem o abaixo:

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, Helvetica, sans-serif;
  background-color: #f5f5f5;
  color: #333333;
  line-height: 1.6;
}

h1 {
  color: #1F4E79;
}

h2 {
  color: #2E75B6;
}

.destaque {
  background-color: #FFF2CC;
  padding: 8px;
}

#sobre {
  background-color: #EBF3FB;
}
```

```
Aula04 > style.css >
1  * {
2    box-sizing: border-box;
3    margin: 0;
4    padding: 0;
5  }
6
7  body {
8    font-family: Arial, Helvetica, sans-serif;
9    background-color: #f5f5f5;
10   color: #333333;
11   line-height: 1.6;
12 }
13
14 h1 {
15   color: #1F4E79;
16 }
17
18 h2 {
19   color: #2E75B6;
20 }
21
22 .destaque {
23   background-color: #FFF2CC;
24   padding: 8px;
25 }
26
27 #sobre {
28   background-color: #EBF3FB;
29 }
```

CSS – Seletores – Prática

Agora dentro do html, escolham algum paragrafo, tag `<p>` para adicionar em sua propriedade `class="destaque"` e vejam como o que o estilo está sendo aplicado na prática, essa é a diferença entre aplicar em um elemento vs o body inteiro.

Senac

CSS – Seletores de combinação

A B

DESCENDENTE

```
main p { }
```

Todo p dentro de main, qualquer nível

A > B

FILHO DIRETO

```
nav > a { }
```

Só a filho imediato de nav

A + B

ADJACENTE

```
h2 + p { }
```

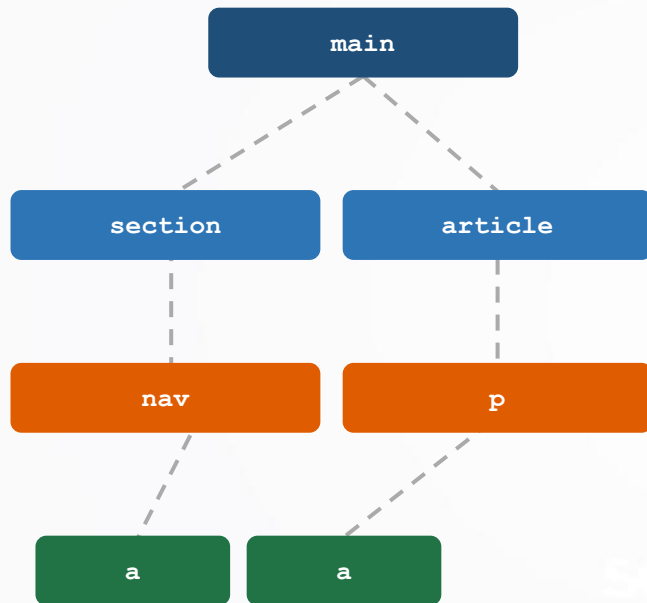
Primeiro p logo após h2

A ~ B

IRMÃO GERAL

```
h2 ~ p { }
```

Todos os p após h2 no mesmo pai



`main p { }` seleciona ambos os `<p>` independente do nível de aninhamento.
`nav > a { }` seleciona apenas os `<a>` filhos diretos de `<nav>`.

CSS – Seletores de combinação

Agora é o momento da prática, acessem o Snippet que se refere ao slide 25, combinadores no portfólio, verifiquem o resultado sobre o arquivo index.



CSS – Pseudo-classes

:hover

Mouse em cima

Botão

:focus

Foco via Tab ou clique

Botão

:active

Sendo clicado

Botão

:visited

Link já visitado

```
a:visited { color: gray; }
```

:first-child

Primeiro filho do pai

```
li:first-child { font-weight: bold; }
```

:not(selector)

Qualquer elemento que NÃO corresponda

```
p:not(.destaque) { color: gray; }
```

:nth-child(even)

Filhos em posições pares

```
tr:nth-child(even) { background: #f0f0f0; }
```

CSS – Pseudo-classes

Agora é o momento da prática, acessem o Snippet que se refere ao slide 27, Pseudo-classes no portfólio, verifiquem o resultado sobre o arquivo index. Notem como ficam os links quando aplica hover, e os inputs quando se aplica a seleção ou a notação via tab. Isso é o CSS sendo aplicado nas pseudo classes do nosso index.



CSS – Especificidade

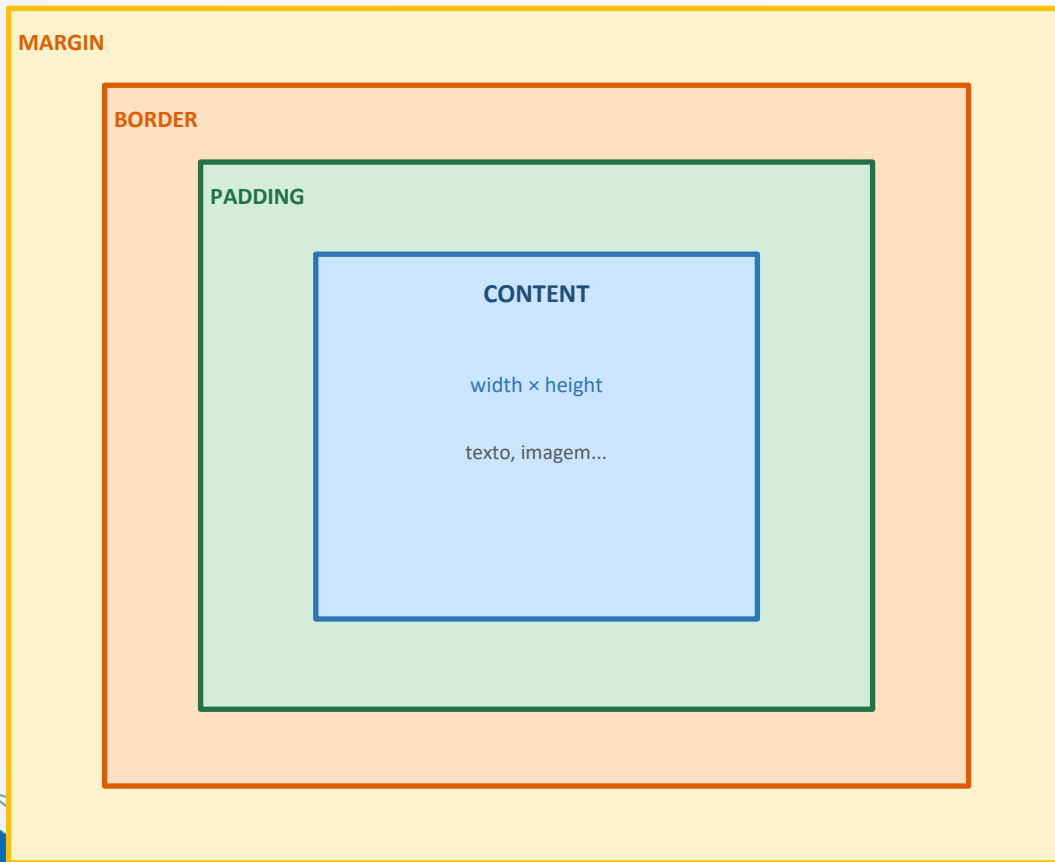
important	inline style	ID	classe	elemento
∞ pontos	1000 pontos	100 pontos	10 pontos	1 pontos
Evite sempre Quebra a cascata	style="..." Evite sempre	#meuId { }	.minhaClasse { }	p { } h2 { }

style.css – Exemplo de cálculo de especificidade

```
p { color: blue; }           /* peso: 1 – elemento */
.texto { color: green; }     /* peso: 10 – classe */
#intro p { color: red; }     /* peso: 101 – ID + elemento → VENCE */
```

→ resultado: **vermelho** (maior especificidade vence)

Box Model – Toda caixa é uma tag com 4 camadas



MARGIN

Espaço externo — afasta o elemento dos vizinhos.
Não tem cor, é sempre transparente.
margin: 16px ou margin: 8px 16px 8px 16px

BORDER

A borda do elemento.
border: 1px solid #CCCCCC
Faz parte do tamanho sem box-sizing: border-box

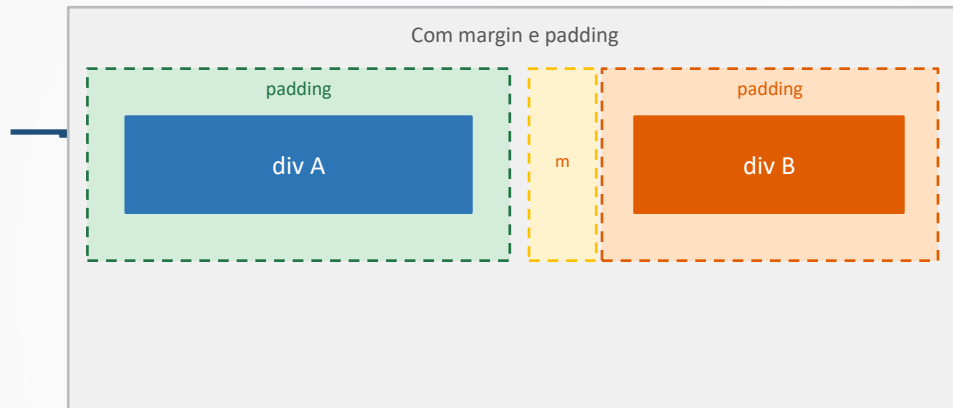
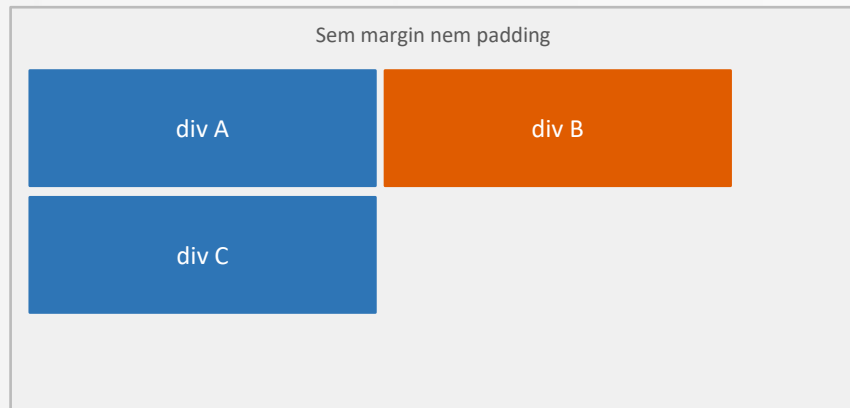
PADDING

Espaço interno — entre conteúdo e borda.
Aceita a cor de fundo do elemento.
padding: 24px ou padding: 8px 16px

CONTENT

O conteúdo real: texto, imagem, vídeo.
Dimensionado por width e height.
Com box-sizing: border-box, é o tamanho final.

CSS – Margin Padding



1 valor — todos os lados

```
margin: 16px;
```

2 valores — cima/baixo, esq/dir

```
margin: 16px 32px;
```

Propriedades individuais

```
margin-top: 8px;  
padding-left: 24px;
```

Centralizar horizontalmente

```
margin: 0 auto; /* requer width */
```

Propriedades individuais

```
margin-top: 8px;  
padding-left: 24px;
```

CSS – Margin Padding

Vamos a prática, acessem o Snippet que se refere ao slide 31, Box Model no portfólio – container e header, sigam os passos. Notem que o resultado já exibe como em um formato web padrão.



CSS – Border e Outline

Estilos de border

solid

```
border: 2px solid #color;
```

dashed

```
border: 2px dashed #color;
```

dotted

```
border: 2px dotted #color;
```

double

```
border: 2px double #color;
```

border-radius

0px

8px

20px

50%

border-radius:

50%

Border vs Outline

border

Faz parte do Box Model
Ocupa espaço no layout
Pode ter espessura diferente por lado

outline

Fica fora do Box Model
Não ocupa espaço no layout
Use para indicar foco em acessibilidade

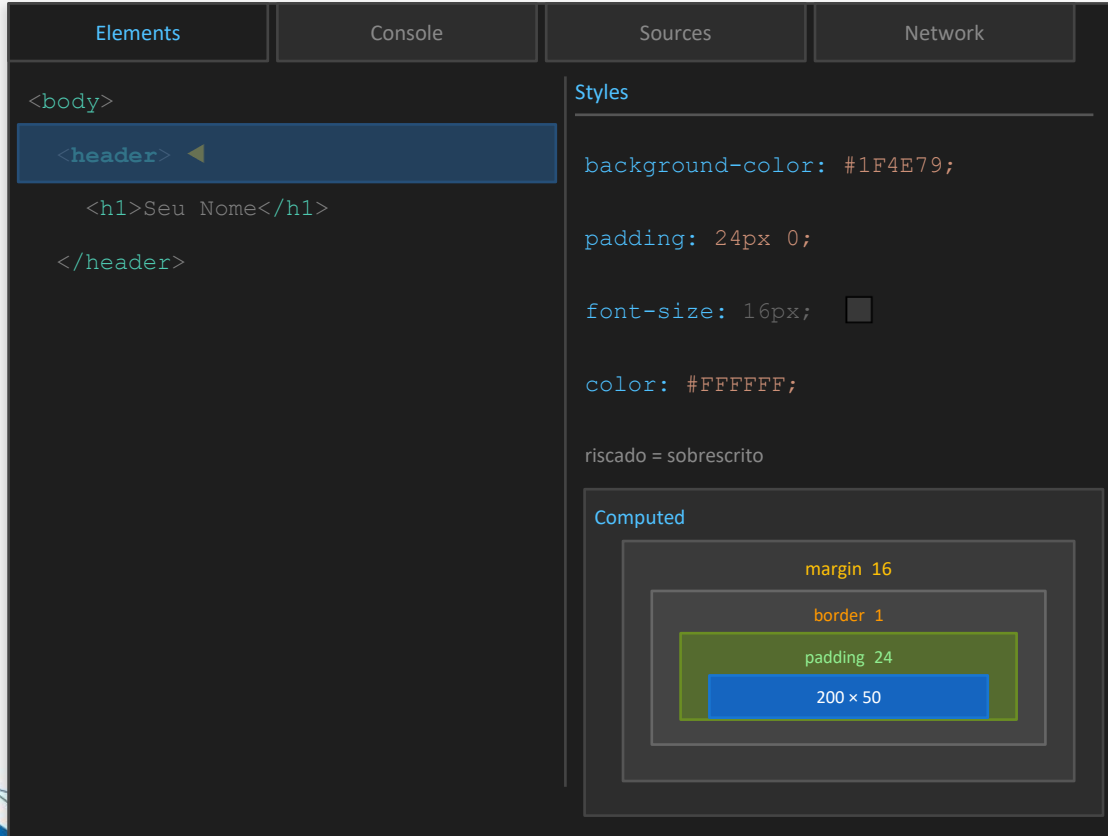
```
input:focus { outline: 2px solid #2E75B6; outline-  
offset: 2px; }  
nunca remova o outline sem substituir por outro  
indicador visual.
```

CSS – Border e Outline

Vamos a prática, acessem o Snippet que se refere ao slide 33, Border, border-radius e foto de perfil circular, sigam os passos. Notem que o resultado já exibe como em um formato web padrão.

Senac

Chrome DevTools



F12

Abrir

F12 ou clique direito → Inspeccionar



Editar

Clique em qualquer valor para editar ao vivo. Mudanças somem ao recarregar, não salvam no arquivo.



Ligar/Desligar

Checkbox ao lado de cada propriedade desativa o estilo sem deletar.



Riscado

Regra riscada = sobrescrita por seletor mais específico. Veja qual regra está vencendo.

Chrome DevTools

Abram o DevTools no portfólio de vocês. Cliquem em qualquer elemento e vejam os estilos aplicados. Encontrem uma propriedade riscada e identifiquem qual regra está vencendo. Usem o painel Computed para ver o Box Model visual do header. Editem uma cor diretamente no DevTools, vejam a mudança, depois apliquem no arquivo CSS.

Senac

CSS – Display: block, inline e híbrido

display: block

div A

div B

div C

Ocupa largura inteira
Quebra linha antes e depois
div, p, h1–h6, section, article

display: inline

span

a

strong

flui junto ao texto sem quebrar linha

width e height não têm efeito

Ocupa só o espaço do conteúdo
Sem quebra de linha
span, a, strong — width e height NÃO funcionam

display: inline-block

btn

btn

btn

Lado a lado + aceita width e height
Combina comportamento inline e block
Botões e ícones com tamanho fixo

display: none → Remove o elemento completamente do layout. O espaço que ele ocuparia some. Diferente de
visibility: hidden que esconde mas mantém o espaço.

CSS – Display: block, inline e híbrido

Novamente vamos a prática, sigam o snippet Slide 37 – Display: inline-block nos links do nav, copiem para o arquivo de styles.css de vocês, vejam o que vai acontecer com o projeto.

Vejam a diferença: antes os links eram texto simples, agora são botões com padding e cor. Isso foi possível porque adicionamos display inline-block, que permite padding e border-radius em elementos que por padrão são inline.

CSS – Unidades de Medida

Unidade	Tipo	O que representa	Melhor para	Exemplo
px	Absoluta	Pixel físico da tela	Borders, shadows, ícones fixos	<code>border: 1px solid #CCC</code>
%	Relativa	Porcentagem do elemento pai	Larguras de container, imagens	<code>width: 100%</code>
em	Relativa	Font-size do elemento atual	Padding e margin relativos à fonte	<code>padding: 1em</code>
rem	Relativa	Font-size do elemento raiz (html)	Tipografia, espaçamentos globais	<code>font-size: 1.5rem</code>
vw	Relativa	% da largura da viewport	Seções full-width, títulos hero	<code>width: 100vw</code>
vh	Relativa	% da altura da viewport	Seções full-screen, hero sections	<code>height: 100vh</code>
ch	Relativa	Largura do caractere O da fonte	max-width de texto para leitura	<code>max-width: 65ch</code>

Boas práticas: tipografia → rem | espaçamentos → rem ou px | largura de container → % | seção hero → vh | max-width de texto → ch

CSS – Unidades de Medida

Px é previsível mas rígido. Rem é o mais recomendado para tipografia porque respeita a configuração de tamanho de fonte do sistema do usuário, o que é acessibilidade real. Vw e vh são ótimos para seções que precisam ocupar a tela inteira. Altura fixa em container de texto é um dos erros mais comuns: quando o texto cresce (usuário aumenta a fonte, ou o conteúdo muda), o container não acompanha e o texto vaza.

CSS – Unidades de Medida

Última prática agora, sigam o snippet Slide 40 – Unidades de medida e tipografia no portfolio, copiem para o arquivo de styles.css de vocês, vejam o que vai acontecer com o projeto.

O max-width em ch é uma unidade interessante: ch equivale à largura do caractere zero da fonte atual. 65ch é aproximadamente o comprimento de linha ideal para leitura confortável, que fica entre 60 e 75 caracteres por linha. Qualquer livro ou artigo bem diagramado usa esse princípio.

O que foi aplicado hoje



Reset universal com `box-sizing: border-box`



Tipografia base no `body` com `rem`



Container centralizado com `margin: 0 auto`



Header estilizado com `padding` e cor de fundo



Links do nav como botões com `display: inline-block`



Sections com `border`, `padding` e `background`



Foto de perfil circular com `border-radius: 50%`



Imagens responsivas com `max-width: 100%`



Footer estruturado



Hover nos links com `:hover`



Focus visível nos campos com `:focus`

Na Aula 05: Google Fonts, variáveis CSS, gradientes, backgrounds e transições. O portfólio começa a ter identidade visual própria.

Tarefa – Aula 04

- 1 Continue estilizando o portfólio aplicando cores, tipografia, espaçamentos e border-radius.
- 2 Adicione ao menos um efeito `:hover` e garanta que o `:focus` dos campos de formulário está visível.
- 3 Abra o DevTools em um site que vocês costumam usar. Tente identificar box model, display e unidades usadas.
- 4 Valide o HTML no W3C Validator. CSS não tem validador obrigatório agora, mas o DevTools mostrará erros de propriedade.

Próxima aula — CSS3: Google Fonts, variáveis CSS, gradientes, backgrounds e transições.